



LINUX FUNDAMENTALS

Samjhana Dahal

Bsc (Hons) Computer Science with Artificial Intelligence

Softwarica College of IT and E-commerce ST5003CMD

Operating Systems, Security, and Networks

Rupak Rajbanshi

9th September 2026

Table of Contents

1.File Operation.....	4
1.1 Creating and Navigation to a lab directory	4
1.2 Create and edit a file	4
1.3 View file content	4
1.4 Copy and Rename	5
1.5 Remove files and directories.....	5
1.6 Check permissions	5
2. APT PACKAGE MANAGEMENT.....	5
2.1 Update the package list	5
2.2 Install the C Compiler	6
2.3 Verify installation	6
2.4 Search for other packages	7
3. C COMPILATION	8
3.1 Write the C code.....	8
3.2 Compile and Run	8
3.3 Check if the executable was created	8
3.4 Run the compiled program.....	9
3.5 Compile with warnings enabled.....	9
4. MODIFY AND RECOMPILE.....	9
4.1 Recompile	10
4.2 Run again to see changes	10
5. CLEANUP AND VERIFICATION	11
5.1 Go back to home directory	11
5.2 List all files in the lab directory with details.....	11
5.3 View the content of your C file one last time	11
5.4 Remove the entire lab directory when done.....	12

Table of figures

Figure 1: Creating and navigating to the linux_lab directory	4
Figure 2: Creating and editing greeting.txt	4
Figure 3: Viewing the content of greeting.txt	4
Figure 4: Copying greeting.txt to backup.txt	5
Figure 5: Removing old_greeting.txt.....	5
Figure 6: Checking the permission of greeting.txt.....	5
Figure 7:Updating the package list	6
Figure 8: Installing build-essential.....	6
Figure 9: Verifying the GCC installation	6
Figure 10:Searching for Python 3 packages	7
Figure 11: Writing the C code using the Nano editor	8
Figure 12: Compiling the hello.c source file using GCC.....	8
Figure 13: Checking the created executable	9
Figure 14: Running the compiled C program	9
Figure 15: Compiling the C program with warnings enabled.....	9
Figure 16: Editing the C Program Using Nano.....	9
Figure 17: Editing the hello.c Source Code Using the Nano Text Editor	10
Figure 18: Recompiling the Modified C Program Using GCC	10
Figure 19: Running the Modified Program to Verify Changes	10
Figure 20: Returning to the Home Directory	11
Figure 21: Listing Files in the Lab Directory with Details	11
Figure 22: Viewing the Final Content of the C Source File.....	11
Figure 23: Removing the Linux Lab Directory	12

1.File Operation

1.1 Creating and Navigation to a lab directory

The mkdir command is used to create a new directory, while cd is used to navigate into a directory. In this practical, the linux_lab directory was created and opened as the working directory for performing the file operations.

```
samjhana@DESKTOP-MDG5KAN:~$ mkdir ~/linux_lab  
samjhana@DESKTOP-MDG5KAN:~$ cd ~/linux_lab
```

Figure 1: Creating and navigating to the linux_lab directory

1.2 Create and edit a file

The nano command is a terminal-based text editor used to create and modify files. In this practical, greeting.txt was created and “**Hello Linux!**” was entered.

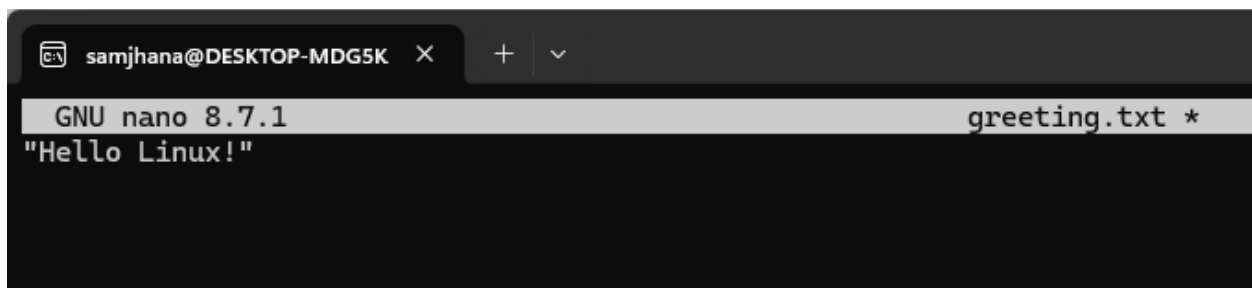


Figure 2: Creating and editing greeting.txt

1.3 View file content

The cat command is used to display the contents of a file directly in the terminal. It was used to verify the content stored in greeting.txt.

```
samjhana@DESKTOP-MDG5KAN:~$ nano greeting.txt  
samjhana@DESKTOP-MDG5KAN:~$ cat greeting.txt  
"Hello Linux!"
```

Figure 3: Viewing the content of greeting.txt

1.4 Copy and Rename

The `cp` command is used to copy a file, while `mv` can be used to rename or move it. In this practical, `greeting.txt` was copied as `backup.txt` and then renamed.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ cp greeting.txt backup.txt
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ mv backup.txt old_greeting.txt
```

Figure 4: Copying greeting.txt to backup.txt

1.5 Remove files and directories

The `rm` command is used to remove files, while `rm -r` removes directories and their contents. In this practical, the unwanted file and temporary directory were removed.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ rm old_greeting.txt
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ mkdir temp_dir
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ rmdir temp_dir
```

Figure 5: Removing old_greeting.txt

1.6 Check permissions

The `ls -l` command displays detailed file information, including permissions, ownership, and file size. It was used to check the permissions of `greeting.txt`.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ ls -l greeting.txt
greeting.txt
```

Figure 6: Checking the permission of greeting.txt

2. APT PACKAGE MANAGEMENT

APT is a package management system used in Ubuntu to manage and install software packages. It also allows users to update package information and search for available packages.

2.1 Update the package list

The `apt update` command refreshes the list of available packages from Ubuntu repositories. This ensures that the system has updated package information.

```

samjhana@DESKTOP-MDG5KAN:~/linux_lab$ sudo apt update
[sudo: authenticate] Password:
sudo: Authentication failed, try again.
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.6 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.4 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [601 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [98.3 kB]
Get:9 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [268 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [188 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [409 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [79.5 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [11.6 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse Translation-en [3036 B]
Get:15 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1056 B]
Fetched 2161 kB in 4s (571 kB/s)
4 packages can be upgraded. Run 'apt list --upgradable' to see them.

```

Figure 7: Updating the package list

2.2 Install the C Compiler

The building-essential package provides the essential tools required for compiling C programs. It was installed to prepare the system for C programming.

```

samjhana@DESKTOP-MDG5KAN:~/linux_lab$ sudo apt install build-essential
build-essential is already the newest version (12.12ubuntu2.26.04.2).
build-essential set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 4

```

Figure 8: Installing build-essential

2.3 Verify installation

The gcc --version command displays the installed version of the GCC compiler. It was used to verify that GCC was correctly installed.

```

samjhana@DESKTOP-MDG5KAN:~/linux_lab$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

samjhana@DESKTOP-MDG5KAN:~/linux_lab$ |

```

Figure 9: Verifying the GCC installation

2.4 Search for other packages

The apt search command is used to find packages available in the configured repositories. In this practical, python3 was searched to find related packages.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ apt search python3
alembic/resolute 1.18.4-1 all
  lightweight database migration tool for SQLAlchemy

antimony/resolute 0.9.3-2.1build2 amd64
  Computer-aided design CAD tool

astro-frameworks/resolute 5.0 all
  Astronomical data analysis frameworks

astro-python3/resolute 5.0 all
  Python 3 packages for astronomy

bamkit/resolute 0.0.1+git20170413.ccd079d-3build1 all
  tools for common BAM file manipulations

beancount/resolute 3.2.0-2build1 all
  Double-entry accounting from text files

berrynet-dashboard/resolute 3.10.2-2build1 all
  deep learning gateway - python3 modules

brlTTY/resolute 6.7-1ubuntu6 amd64
  Access software for a blind person using a braille display

budgie-app-launcher-applet/resolute-updates 2.2.3-1ubuntu1.1 amd64
  Applet to provide an alternative means to launch applications

budgie-brightness-controller-applet/resolute-updates 2.2.3-1ubuntu1.1 amd64
  Applet to control the brightness of the screen

budgie-clockworks-applet/resolute-updates 2.2.3-1ubuntu1.1 amd64
  Applet to display clock across multiple time zones

budgie-countdown-applet/resolute-updates 2.2.3-1ubuntu1.1 amd64
  Applet providing a countdown capability on the Budgie Desktop

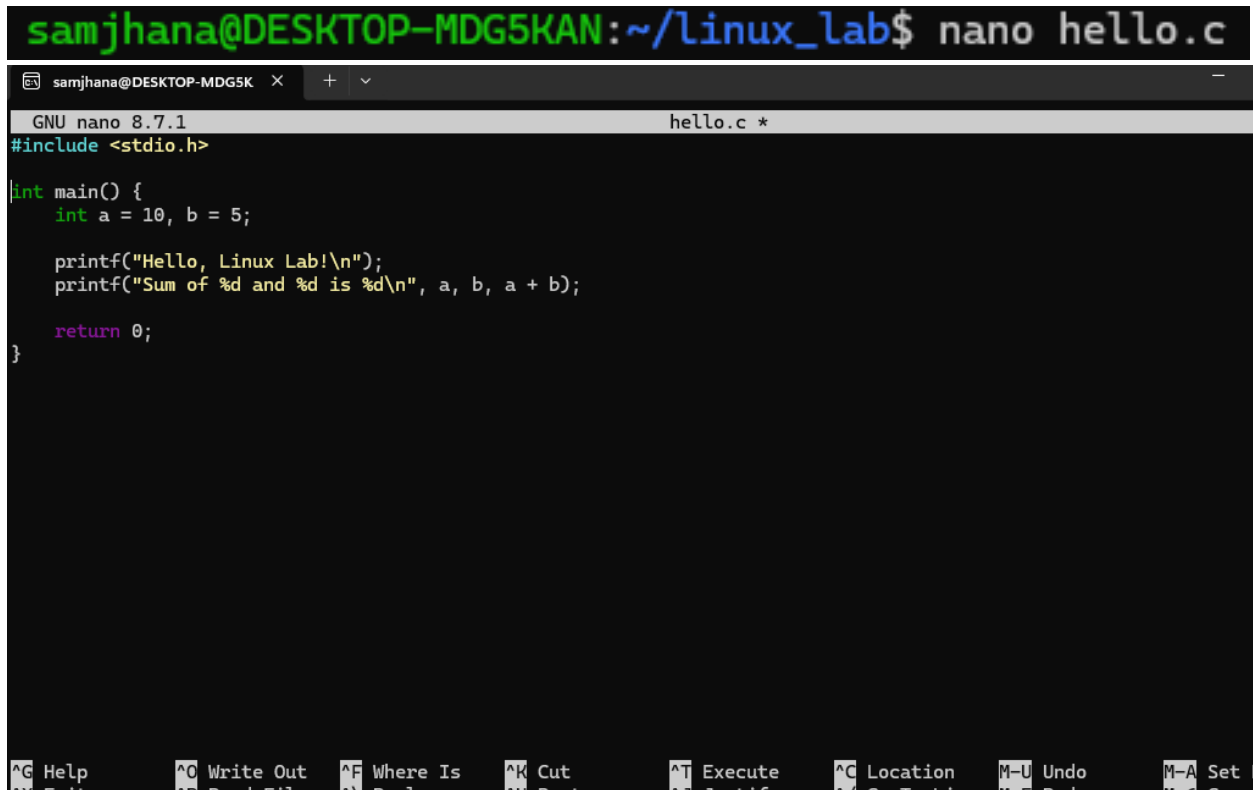
[1]+  Stopped                  apt search python3
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ |
```

Figure 10: Searching for Python 3 packages

3. C COMPILATION

3.1 Write the C code

The nano hello.c command was used to create and edit the C source file. The program displays “Hello, Linux Lab!” and calculates the sum of two integer values, 10 and 5.

A screenshot of a terminal window showing the Nano text editor. The terminal prompt is 'samjhana@DESKTOP-MDG5KAN:~/linux_lab\$'. The command 'nano hello.c' has been executed. The Nano editor window shows the file 'hello.c' with the following C code:

```
#include <stdio.h>

int main() {
    int a = 10, b = 5;

    printf("Hello, Linux Lab!\n");
    printf("Sum of %d and %d is %d\n", a, b, a + b);

    return 0;
}
```

The Nano editor interface includes a title bar 'GNU nano 8.7.1' and a status bar at the bottom with various keyboard shortcuts like '^G Help', '^O Write Out', '^F Where Is', '^K Cut', '^T Execute', '^C Location', '^U Undo', and '^M Set I'.

Figure 11: Writing the C code using the Nano editor

3.2 Compile and Run

The gcc command is used to compile the hello.c source file into an executable program. The -o option names the output file as hello_program.

A screenshot of a terminal window showing the command 'gcc hello.c -o hello_program' being executed. The terminal prompt is 'samjhana@DESKTOP-MDG5KAN:~/linux_lab\$'.

Figure 12: Compiling the hello.c source file using GCC

3.3 Check if the executable was created

The ls command is used to list files and directories. Here, ls hello_program checks whether the compiled executable hello_program was successfully created.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ ls -l hello_program
-rwxr-xr-x 1 samjhana samjhana 15360 Aug 10 10:10 hello_program
```

Figure 13: Checking the created executable

3.4 Run the compiled program

The `./hello_program` command is used to run the executable file created by GCC. The `./` indicates that the program is located in the current directory.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ ./hello_program
Hello, Linux Lab!
Sum of 10 and 5 = 15
```

Figure 14: Running the compiled C program

3.5 Compile with warnings enabled

The `-Wall` option enables a broad set of common compiler warnings when compiling the C program. It helps identify potential issues in the source code.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ gcc -Wall hello.c -o hello_program_warn
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ ./hello_program_warn
Hello, Linux Lab!
Sum of 10 and 5 = 15
```

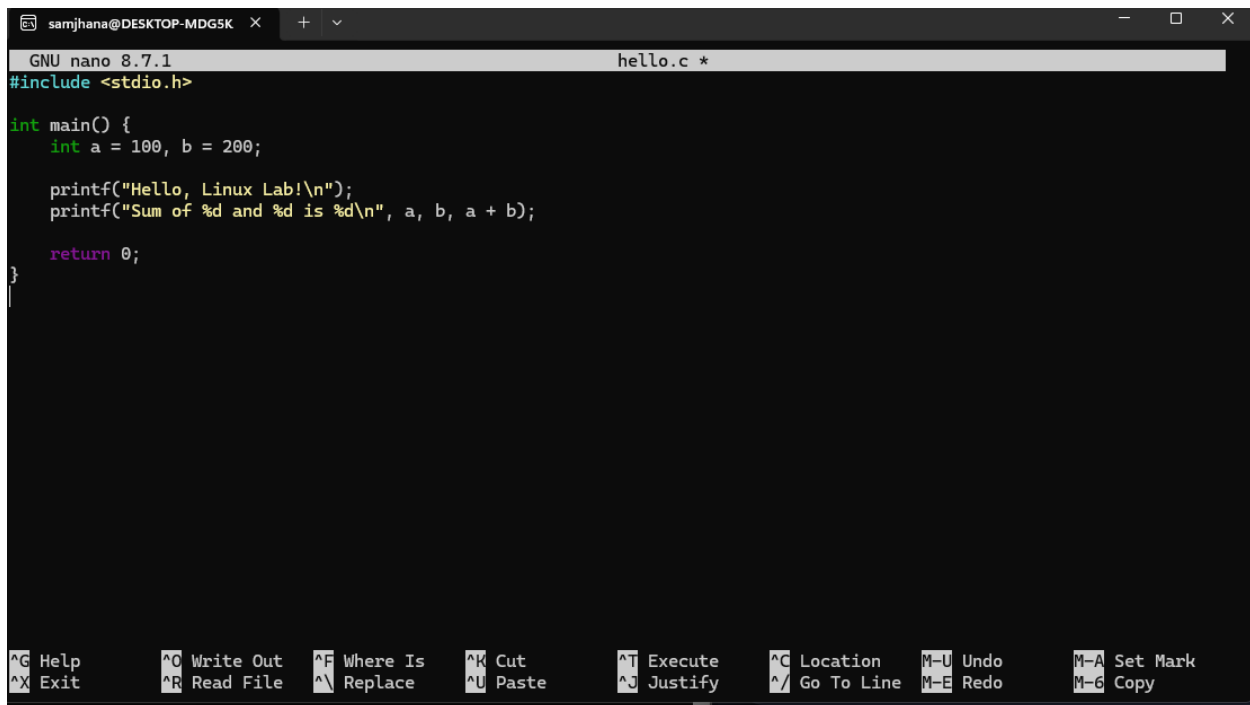
Figure 15: Compiling the C program with warnings enabled

4. MODIFY AND RECOMPILE

`nano` is a text editor used to open or create a file in the terminal. Here, it is used to open `hello.c` so that the C program can be edited or modified.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ nano hello.c
```

Figure 16: Editing the C Program Using Nano



```
GNU nano 8.7.1 hello.c *
#include <stdio.h>

int main() {
    int a = 100, b = 200;

    printf("Hello, Linux Lab!\n");
    printf("Sum of %d and %d is %d\n", a, b, a + b);

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_/ Go To Line M-E Redo M-G Copy

Figure 17: Editing the hello.c Source Code Using the Nano Text Editor

4.1 Recompile

This command recompiles the modified hello.c program using GCC and creates an updated executable file named hello_program

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ gcc -Wall hello.c -o program
```

Figure 18: Recompiling the Modified C Program Using GCC

4.2 Run again to see changes

This command runs the compiled program again to check whether the modifications made to the source code are reflected in the output.

```
samjhana@DESKTOP-MDG5KAN:~$ ./hello_program
Hello, Linux Lab!
Sum of 100 and 200 is 300
samjhana@DESKTOP-MDG5KAN:~$
```

Figure 19: Running the Modified Program to Verify Changes

5. CLEANUP AND VERIFICATION

5.1 Go back to home directory

This command changes the current working directory back to the user's home directory.

```
samjhana@DESKTOP-MDG5KAN:~/linux_lab$ cd ~
```

Figure 20: Returning to the Home Directory

5.2 List all files in the lab directory with details

This command displays all files in the linux_lab directory with detailed information such as permissions, ownership, file size, and modification time.

```
samjhana@DESKTOP-MDG5KAN:~$ ls -lh ~/linux_lab/
total 56K
-rw-r--r-- 1 samjhana samjhana 21 Sep  8 06:15 greeting.txt
-rw-r--r-- 1 samjhana samjhana 162 Sep  8 15:36 hello.c
-rwxr-xr-x 1 samjhana samjhana 16K Sep  8 14:31 hello_program
-rwxr-xr-x 1 samjhana samjhana 16K Sep  8 14:42 hello_program_warn
-rwxr-xr-x 1 samjhana samjhana 16K Sep  8 15:41 program
```

Figure 21: Listing Files in the Lab Directory with Details

5.3 View the content of your C file one last time

This command displays the final contents of the hello.c source file in the terminal for one last verification.

```
#include <stdio.h>

int main(){
    int a = 100, b = 200;

    printf("Hello, Linux Lab!\n");
    printf("Sum of %d and %d is %d.\n", a, b, a + b);
    return 0;
}
```

Figure 22: Viewing the Final Content of the C Source File

5.4 Remove the entire lab directory when done

The `rm -rf` command is used to **remove the linux_lab directory and all files inside it**.

A terminal window with a black background and green text. The prompt is 'samjhana@DESKTOP-MDG5KAN:~\$' and the command entered is 'rm -rf ~/linux_lab'.

```
samjhana@DESKTOP-MDG5KAN:~$ rm -rf ~/linux_lab
```

Figure 23: Removing the Linux Lab Directory